

CS F407 Artificial Intelligence - Week 3: Search and A*

Advay Matapurkar | ID: 2023A7PS0459G

Laboratory report. Using an LLM as an engineering assistant. LLM: Claude via Claude Code. Submitted files: `astar_llm.py` (final program, LLM-generated, unmodified), `test_search.py` (test harness, written by the LLM to my validation plan) and this report, which contains the design, the prompts (Appendix A) and the full test output (Appendix B). Reproduce with `python test_search.py` (Python 3, standard library only; random tests use seed 407).

1. Formulation of the search problem (Task 0)

Component	Specification
State S	Robot position (row, col) on the 9x17 grid; the cell is not '#'.
Actions A	Up, Down, Left, Right (one cell).
Transition T	(r,c) + (dr,dc) with Up=(-1,0), Down=(1,0), Left=(0,-1), Right=(0,1); allowed only if the target is inside the grid and not '#'.
Initial state s0	The cell marked S, (1,1).
Goal G	The cell marked G, (7,15). Goal test: state == goal.
Cost c	1 per move; path cost = number of moves.

(a) Only the robot's position; the map never changes, so it is part of the problem, not the state. (b) An action is invalid if it leaves the grid or enters '#'. (c) Yes, deterministic: each action has exactly one outcome and the map is fully known and static. (d) A sequence of valid actions from S to G; an optimal solution has the fewest moves.

2. Design of the agent (Task 1, written before any code)

Question	Design decision
1. State in Python	A tuple (row, col).
2. Warehouse	A list of lists of characters from the ASCII map, in a Warehouse class that also finds S and G and checks there is exactly one of each.
3. Valid actions	<code>successors(state)</code> tries Up, Down, Left, Right in that fixed order and yields (action, next_state, cost) for legal moves.
4. Goal recognition	<code>is_goal(state)</code> , applied when a node is popped from the frontier (my choice: this preserves optimality; testing at generation time can return a non-optimal path).
5. Frontier contents	A*: heap of (f, counter, state), $f = g + h$; ties broken first-in-first-out by the counter (my choice: the most intuitive rule). Dicts <code>g[state]</code> and <code>parent[state] = (previous state, action)</code> ; a closed set. BFS: FIFO queue plus parent dict.
6. Path reconstruction	Follow parent links from the goal to the start, then reverse.
7. Repeated states	Skip a popped state already in the closed set; push a neighbour only if its new g is lower than the best known g.
8. Heuristic	Manhattan distance, passed to A* as a function so $h=0$, Euclidean and 2x Manhattan can be swapped in (Task 6).
9. Reported on termination	Solution found, path and actions, path length, states expanded. The goal is not counted as expanded (my choice: a state is expanded only when its successors are generated).
10. Validation plan	The four lab tests; comparison of A* with BFS and with an independent oracle; an independent path-replay checker; BFS/A* comparison; heuristic variants.

How the design was produced: I wrote this formulation and design myself, before using the LLM. The design was fixed before the LLM prompt was written, and the prompt before any code.

3. Final Python program (Task 2)

Prompt (full text in Appendix A): I wrote a Task 2-style prompt containing the map and the ten design points above. It was given, on its own, to a fresh LLM instance with no access to any earlier code, which returned `astar_llm.py` (164 lines). I used it **unmodified**. It contains: Warehouse, the four heuristic functions, `astar`, `bfs`, `reconstruct` and `draw/main`. The generator did not run its own code; I ran and tested it (next section). Separately, `test_search.py` (written by the LLM from my test-harness prompt, also in Appendix A; I ran it) holds the test maps, an independent path-replay checker and an independent shortest-path oracle.

4. Test results (Task 3)

Test 1 - original warehouse (A*, Manhattan). Path found: yes. Length: 40. States expanded: 63. Path replay: valid; independent oracle length: 40.

```
#####
#S*****#
#.#*#####*#
#...#*#####*#
###.#*#####*#
#...#*#####*#
#.#*#####*#*#
#.#*#####*#*#
#.....#G#
#####
```

Path (row,col): (1,1) (1,2) (1,3) (1,4) (1,5) (2,5) (3,5) (4,5) (5,5) (5,6) (5,7) (5,8) (5,9) (5,10) (5,11) (5,12) (5,13) (4,13) (3,13) (3,12) (3,11) (3,10) (3,9) (3,8) (3,7) (2,7) (1,7) (1,8) (1,9) (1,10) (1,11) (1,12) (1,13) (1,14) (1,15) (2,15) (3,15) (4,15) (5,15) (6,15) (7,15)

Test 2 - trivial case (#####/#SG##/#####). A*: found=True, length=1, expanded=1. Correct: one step, Right.

Test 3 - no solution. Goal walled off. A*: found=False after expanding 9 states (BFS: found=False, 9). The program terminates and reports failure; the closed set guarantees this because the state space is finite.

Test 4 - alternative paths. The top row is a length-4 route; the bottom loop is length 8.

```
#####
#S...G#
#...#.#
#...#.#
#...#.#
#...#.#
#####
```

A* returned length 4 (expanded 4); BFS returned length 4 (expanded 8); the independent oracle says the shortest is 4. Equal, so A* returned a shortest path.

Extra validation. On 500 random 9x13 grids (298 solvable, 202 unsolvable) I compared each result with the independent oracle (length, found/not found, and path replay). Disagreements: Manhattan 0, h=0 0, Euclidean 0, BFS 0, 2xManhattan 85. So A* with an admissible heuristic was checked against an independent method on many maps, not just one.

5. Where each concept appears in the code (Task 4, `astar_llm.py`)

Concept	Location
State	(row, col) tuples yielded by Warehouse.successors (lines 36-42); start/goal found in Warehouse.__init__ (19-31).
Action	MOVES list (15) and the loop in successors (38).
Transition	nr, nc = r + dr, c + dc (line 39), after the bounds and '#' check (40-41).
Goal test	Warehouse.is_goal (33), called at line 97 in astar.
g(n)	dict g (89); new_g = g[state] + cost (103); stored at 105.
h(n)	h_zero / h_manhattan / h_euclidean / h_double_manhattan (46-59); called at 88 and 109.
f(n)	new_g + heuristic(nxt, goal) at line 109, stored as the first element of the heap tuple (start node: line 88).
Frontier	heapq list 'frontier' (88); pushed at 108-109, popped at 94.

Visited states	closed set (91); checked at 95, added at 100. The g dict also stops worse duplicates being pushed (104).
Path reconstruction	parent dict (90, 106) and reconstruct (62-72).

(a) A binary min-heap (Python heapq) of (f, counter, state) tuples. **(b)** heappop returns the entry with the smallest f; ties go to the smaller counter, i.e. the entry inserted first. **(c)** In the heuristic functions (lines 46-59), invoked when nodes are pushed (lines 88, 109). **(d)** Yes: f is computed as $\text{new_g} + \text{heuristic}(\text{nxt}, \text{goal})$ at each push and used as the heap key, rather than kept in a separate variable. **(e)** Two mechanisms: the closed set (an already-expanded state is skipped when popped, which also discards stale duplicates) and the g dict (a neighbour is pushed only if this route to it is cheaper than any known one).

6. BFS vs A* (Task 5), same warehouse

Measure	BFS	A* (Manhattan)
Solution found	True	True
Path length	40	40
States expanded	63	63

(a) Yes, both. **(b)** Yes, both length 40. **(c)** Neither: both expanded 63 states. **(d)** In general A* can expand fewer states because it uses the goal location, preferring nodes with low estimated total cost instead of exploring in rings around the start. Other maps show this: in Test 4 A* expanded 4 states against BFS's 8, and on 456 solvable random open grids (15% walls) A* with Manhattan expanded a mean 81.7 states against 98.2 for BFS. The warehouse itself is a winding corridor (64 free cells) whose only route to G first heads away from it (right along the top, then down). Manhattan distance points the wrong way for much of that route, so every cell has $f = g + h$ at most the optimal cost 40 (I checked: the maximum is 40, reached by 14 cells). A* must expand all nodes with f below the optimal cost and, through ties, also reached the rest: all 63 non-goal free cells were expanded. This is the lab's point: A* is not automatically better, its gain depends on how well the heuristic matches the map.

7. Heuristic investigation (Task 6)

Why Manhattan suits this problem. The robot moves only horizontally and vertically at cost 1, so any path from n to the goal needs at least $|dx| + |dy|$ moves. The estimate never exceeds the true cost (admissible) and changes by at most 1 per move (consistent), hence A* returns optimal paths. The experiments below check this rather than relying on the argument.

Heuristic	Found	Path length	States expanded (warehouse)	Mean expanded, 456 open grids	Mean path length, open grids
Manhattan	True	40	63	81.7	20.02
Zero (h=0)	True	40	63	98.2	20.02
Euclidean	True	40	63	87.3	20.02
2 x Manhattan	True	40	63	25.5	21.00
BFS (reference)	True	40	63	98.2	20.02

h = 0. A* becomes uniform-cost search (same as BFS with unit costs): still optimal, but expands the most (98.2 on average on open grids). **Euclidean.** Never larger than Manhattan, so still admissible and optimal, but less informed, so it expands more (87.3 vs 81.7). **2 x Manhattan.** It overestimates (not admissible). On the warehouse it gave the same path, because the corridor offers no alternative, but on open grids it expanded far fewer states (25.5) and returned longer paths (21.00 vs 20.02 on average). On the 500 random grids of the correctness check it returned a non-shortest path 85 times. Example (shortest is 20, 2xManhattan gave 24):

```
S.....
.#.....
...#.....
.....#.....#
.#.....#.
```

```
#.....#.#.#.
...##.....#.
...#.....#....
#.....#...G
```

Conclusion. A weak heuristic ($h=0$) keeps optimality but searches more. An over-aggressive one (h above the true cost) makes A* greedy: it searches less and can miss the shortest path. Admissibility, $h(n) \leq h^*(n)$, is what protects against this. On the given warehouse none of the heuristics changed anything, so Test 4 and the random grids were needed to see any effect.

8. Evaluation of the LLM-generated agent (Task 7)

Who did what. *Designed by me:* the problem formulation and the agent design, including the three choices with my own reasons (goal test at pop time to preserve optimality; FIFO tie-breaking as the most intuitive; goal not counted as expanded), and the Task 2 prompt and the test-harness prompt. *Generated by the LLM:* `astar_llm.py`, from the prompt alone, and the test harness `test_search.py`, from my test-harness prompt. *Accepted:* all of `astar_llm.py`. *Changed:* nothing in `astar_llm.py`. *Tested by me:* I ran the four lab tests, path replay, the independent oracle on 500 random grids, BFS vs A*, and the heuristic variants, and checked the results.

Question	Answer
1. Correct immediately	Everything I tested: A* and BFS returned valid, shortest paths on all four tests and on 500 random grids for the admissible heuristics; unsolvable maps terminated; all ten design points were followed, including the pop-time goal test, the FIFO counter and not counting the goal.
2. Bugs / design problems	No bug found. Observations: (i) the BFS keeps both a 'closed' set and a 'seen' set; the closed set and its check are redundant because the seen set already stops duplicates (harmless); (ii) the generator did not run its own code, so the first execution was mine; (iii) A* with a closed set and no re-opening is only guaranteed optimal for consistent heuristics, which showed with 2xManhattan (85 of 500 grids not shortest); (iv) the given warehouse cannot distinguish the algorithms.
3. How discovered	By running the tests: path replay and the independent oracle, the random grids, and noticing that every heuristic gives 40 / 63 on the warehouse.
4. Unfamiliar terms	Nothing unfamiliar in the code. Worth understanding: the heap tie-breaking counter (equal-f tuples would otherwise compare states) and stale entries (duplicates left in the heap, skipped via the closed set).
5. Modified?	No. The program was used exactly as generated.
6. Most useful tests	The random comparison with the independent oracle (it exposed the 2xManhattan failures) and the no-solution test (termination). The trivial test caught nothing but confirmed the base case.
7. Trust without testing?	No. On the warehouse alone, a wrong heuristic gives exactly the same output as the right one, so that test could not reveal a heuristic bug; and the code had never been executed by its author before I ran it.
8. Learned about A*	Its advantage depends on the map and the heuristic (identical on the corridor, clearly fewer expansions on open grids); the goal test belongs at pop time; and admissibility, not just a plausible estimate, is what guarantees shortest paths.

9. Final reflection (Section 6)

1. Why formulate the problem first? The formulation fixes what a state is, which actions are legal and what counts as a solution; the algorithm is generic over those. With them written down, the code is a translation of a specification, testing has concrete criteria (legal moves, goal reached, cost), and the LLM's output can be checked against something. In this lab the ten design points became the prompt, and I could check the generated code against each of them.

2. Informed search. A* uses knowledge beyond the problem definition: the heuristic $h(n)$, an estimate of the remaining cost. It orders the frontier by $f = g + h$, combining the cost paid so far with the estimated cost to go, whereas BFS orders only by discovery order (depth).

3. Why the heuristic matters. It determines both efficiency and correctness. $h=0$ is safe but uninformed; Manhattan is admissible and informed on a 4-connected grid; 2xManhattan is more aggressive and expanded

about a quarter of the states of $h=0$ on open grids (25.5 against 98.2), but returned non-shortest paths on 85 of 500 test grids. On the corridor-like warehouse, no heuristic helped at all.

4. What the LLM contributed. A complete, correct implementation from a precise prompt, quickly: data structures, the priority-queue idiom, BFS for comparison and the heuristic variants. It did not replace judgement: deciding the design (what to test, when to test the goal, how to break ties), interpreting why the warehouse results were identical, and understanding the 2xManhattan failures remained the engineer's job. A clear design also made the prompt precise, which is likely why the first output needed no changes.

5. Accepting code untested. Plausible but wrong code goes unnoticed: paths that cross walls, infinite loops on unsolvable maps, non-optimal paths from an inadmissible heuristic, or code that is right on one map and wrong elsewhere (the warehouse output alone looks right for every heuristic). Testing against known answers and an independent algorithm is what turns working output into a validated algorithm.

Appendix A: Prompts used

PROMPTS USED WITH THE LLM (Claude, via Claude Code)

Process order actually followed

1. Read the lab sheet (search_lab.pdf).
2. I wrote the Task 0 formulation and the Task 1 design, including its three choices (goal test at pop time, FIFO tie-breaking, goal not counted as expanded).
3. I wrote the Task 2 prompt (below) from that design.
4. The prompt was given, on its own, to a fresh LLM instance (a Claude subagent with no access to earlier code). Its only extra instructions were: do not read existing files, and save the program to a given path. It returned `astar_llm.py`; I copied that file unmodified.
5. I wrote the test-harness prompt (below) from the validation plan in my design. The LLM wrote `test_search.py` from it; I ran it (output in Appendix B).

Task 2 prompt (given to the code-generating LLM; full text)

I am implementing a simple goal-based search agent in Python 3 (standard library only).

The environment is a grid given as an ASCII map. The agent starts at S and must reach G. '#' is an obstacle and '.' is a free cell. The agent moves Up, Down, Left or Right, one cell at a time, and every move costs 1. The map is:

```
#####  
#S...#.....#  
#...#.....#  
#...#.....#  
###.#.....#  
#...#.....#  
#.....#G#  
#####
```

My design (please follow it):

1. A state is a tuple (row, col). The map is part of the problem, not of the state.
2. A Warehouse class holds the grid (list of lists of characters), finds S and G (checking there is exactly one of each), and provides `is_goal(state)` and `successors(state)`.
3. `successors(state)` tries the moves in the fixed order Up, Down, Left, Right and yields (action, next_state, cost) for moves that stay inside the grid and do not enter '#'.
4. Goal test: apply `is_goal` when a node is POPPED from the frontier, not when it is generated, so that A* stays optimal.
5. Implement A* search taking the heuristic as a function argument. Frontier: a heapq priority queue of (f, counter, state) with $f = g + h$. The counter is an increasing integer so ties in f are broken first-in-first-out. Keep dicts `g[state]` and `parent[state] = (previous_state, action)`, and a closed set of expanded states.
6. Avoid repeated expansion: skip a popped state that is already in the closed set, and push a neighbour only if its new g is lower than the best known g for it.
7. Heuristic: Manhattan distance $|r - r_{goal}| + |c - c_{goal}|$. Also provide, as interchangeable functions, $h = 0$, Euclidean distance, and $2 \times$ Manhattan.
8. Also implement breadth-first search (FIFO queue) for the same problem, so the two can be compared.
9. Reconstruct the path by following parent links from the goal back to the start.
10. Both searches must return: whether a solution was found, the path (states and actions), the path length, and the number of states expanded. Count a state as expanded only when it is popped, is NOT the goal and its successors are generated; the goal itself is not counted.

Provide a `main()` that runs A* (Manhattan) on the map above and prints the results and the map with the path drawn. Keep the implementation simple and explain the main components of the code briefly. Do not assume the code is correct; I will test it.

Test-harness prompt (written by me and given to the LLM once `astar_llm.py` existed; full text)

I have a Python 3 program, `astar_llm.py` (standard library only), that implements A* and BFS on an ASCII warehouse grid. I need you to write a test harness for it, as a single file `test_search.py`. Do not modify `astar_llm.py`; import it with ``import astar_llm as L``.

Interface of `astar_llm.py`:

- `L.MAP`: the warehouse map string (9 rows x 17 columns; '#' obstacle, '.' free, one 'S', one 'G').
- `L.Warehouse(text)`: builds a problem from a map string. Attributes: `.grid` (list of lists of characters), `.start` and `.goal` as (row, col) tuples. Methods: `is_goal(state)`, `successors(state)` yielding (action, next_state, cost).
- Heuristic functions taking (state, goal): `L.h_manhattan`, `L.h_zero`, `L.h_euclidean`, `L.h_double_manhattan` (2 x Manhattan).
- `L.astar(problem, heuristic)` and `L.bfs(problem)`. Both return a dict with keys: "found" (bool), "states" (list of (row, col) from start to goal, [] if none), "actions" (list of "Up"/"Down"/"Left"/"Right"), "length" (int, or None if not found), "expanded" (int).
- `L.draw(problem, states)`: returns the map as a string with the path marked by '*'.

Moves are Up=(-1,0), Down=(1,0), Left=(0,-1), Right=(0,1); every move costs 1.

The harness must, when run with ``python test_search.py``, print a clearly labelled report (reconfigure stdout to UTF-8) containing:

1. Test 1: the original warehouse (`L.MAP`), A* with Manhattan. Print found, path length, states expanded, the path as (row,col) coordinates, the action list, and the map with the path drawn.
2. Test 2 (trivial): map "#####\n#SG##\n#####". Expect found, length 1.
3. Test 3 (no solution): map

#S...#
###.###
#...G#

Expect found=False and that the program terminates.
4. Test 4 (alternative paths): map

#S...G#
#.###.#
#....#

Expect a shortest path of length 4.
For tests 2-4 print the results for both A* (Manhattan) and BFS.
5. An independent shortest-path oracle: your own BFS written inside `test_search.py` that works on the raw grid and does NOT call `astar_llm`'s search functions. Use it to check the length reported by A* and BFS on every test.
6. An independent path checker: replay the returned actions from the start, checking that every move is legal (in bounds, not '#'), the states match the actions, the path ends on the goal, and the reported length equals the number of actions. For an unsuccessful result it should check that `states == []` and length is None.
7. Task 5 comparison: on the original warehouse print a table (Measure, BFS, A*) with: solution found, path length, states expanded.
8. Task 6 heuristic investigation: on the original warehouse, run A* with Manhattan, $h=0$, Euclidean and 2 x Manhattan and print a table with: found, path length, states expanded, and whether the path is valid.
9. Randomised validation: use `random.Random(407)`. Generate 500 random 9x13 grids (each cell '#' with probability 0.28, otherwise '.'); put 'S' at the top-left cell and 'G' at the bottom-right cell). For each grid, compare A* with each of the four heuristics, and BFS, against the oracle (length, including "no path" = None, and path validity). Print how many grids were solvable/unsolvable and, for each algorithm/heuristic, how many grids disagreed with the oracle.
10. Open-grid statistics: with a fresh `random.Random(407)`, generate 500 grids of the same size with wall probability 0.15. On the solvable ones, print a table of the mean states expanded and mean path length for A* with each of the four heuristics and for BFS. Also print one example grid where 2 x Manhattan returns a path longer than the oracle's shortest, with both lengths.

Requirements: standard library only; deterministic output; readable section headings such as "=== Test 1: original warehouse ==="; put the test maps in module-level constants; end with ``if __name__ == "__main__": main()`. Keep the code simple, and explain the main parts of it briefly after the code. Do not assume your code is correct; I will run it.`

(Long lines of both prompts are wrapped here to fit the page; continuation lines are indented.)

Appendix B: Full test output (python test_search.py)

```
=== Test 1: original warehouse (A*, Manhattan) ===
found=True length=40 expanded=63 | valid: True | oracle length: 40
Path: (1,1) (1,2) (1,3) (1,4) (1,5) (2,5) (3,5) (4,5) (5,5) (5,6) (5,7) (5,8) (5,9) (5,10) (5,11) (5,12) (5,13)
      (4,13) (3,13) (3,12) (3,11) (3,10) (3,9) (3,8) (3,7) (2,7) (1,7) (1,8) (1,9) (1,10) (1,11) (1,12) (1,13)
      (1,14) (1,15) (2,15) (3,15) (4,15) (5,15) (6,15) (7,15)
Actions: Right, Right, Right, Right, Down, Down, Down, Down, Right, Right, Right, Right, Right, Right, Right,
        Right, Up, Up, Left, Left, Left, Left, Left, Up, Up, Right, Right, Right, Right, Right, Right, Right,
        Right, Down, Down, Down, Down, Down, Down
#####
#S*****
#.#*#####
#...#*#####
###.#*#####
#..#*#####
#.#####.*#
#.....#G#
#####

=== Test 2: trivial case (expected: found, length 1) ===
#####
#SG##
```

```

#####
A*: found=True length=1 expanded=1 | valid: True
BFS: found=True length=1 expanded=1 | valid: True
Independent oracle length: 1 | A* matches oracle: True

=== Test 3: no solution (expected: not found, terminates) ===
#####
#S...#
###.###
#...#G#
#####
A*: found=False expanded=9 | valid: True
BFS: found=False expanded=9 | valid: True
Independent oracle length: None | A* matches oracle: True

=== Test 4: alternative paths (expected: found, shortest = 4) ===
#####
#S...G#
#...#.#
#....#
#####
A*: found=True length=4 expanded=4 | valid: True
BFS: found=True length=4 expanded=8 | valid: True
Independent oracle length: 4 | A* matches oracle: True

=== Task 5: BFS vs A* on the warehouse ===
Measure      BFS      A*
Solution found  True     True
Path length    40      40
States expanded 63      63

=== Task 6: heuristic investigation (warehouse) ===
Heuristic      Found Length Expanded
Manhattan      True   40     63   valid=True
Zero (h=0)     True   40     63   valid=True
Euclidean      True   40     63   valid=True
2 x Manhattan  True   40     63   valid=True

=== Validation: 500 random 9x13 grids vs independent oracle ===
solvable=298 unsolvable=202
Grids where the result differs from the oracle (length/validity):
Manhattan      0
Zero (h=0)     0
Euclidean      0
2 x Manhattan  85
BFS            0

=== Open grids (9x13, 15% walls): mean over solvable grids ===
solvable grids: 456
Algorithm/h      Mean expanded Mean length
Manhattan        81.7      20.02
Zero (h=0)       98.2      20.02
Euclidean        87.3      20.02
2 x Manhattan    25.5      21.00
BFS              98.2      20.02

Example where 2 x Manhattan is NOT shortest (shortest 20, got 24):
S.....
.#.....
...#.....
.....#....#
.#.....#.#
#.....#.#.
...##.....#
...#.....#
#.....#..G

```